

Edge-Efficient, Ethically-Constrained Spatiotemporal Hypergraph Modeling for Urban Forecasting and Decision Support

Phase 1 Revision (Complete Rewrite)

Version: 2025-12-27

Abstract

We present an end-to-end framework for real-time urban sensing, forecasting, and decision support that is **edge-efficient**, **scalable**, and **ethically constrained**. The system converts streaming video and sensor inputs into a compact, privacy-preserving **urban state** representation on the edge, then performs forecasting and counterfactual scenario analysis using a **spatiotemporal hypergraph world model**. Ethical requirements (privacy, policy compliance, and fairness) are expressed as **explicit constraints** enforced during learning and inference via a primal-dual optimization procedure. For teams pursuing a quantum component, we describe an **optional hybrid quantum kernel** used narrowly within hyperedge attention/similarity, with a strict ablation plan and no claims of guaranteed advantage.

This revision replaces ad-hoc numerological invariants with a falsifiable, auditable, and reproducible methodology aligned with deployment and evaluation requirements.

Keywords: smart cities, edge AI, hypergraph neural networks, constrained optimization, counterfactual simulation, privacy, fairness, hybrid quantum kernels

1. Introduction

Cities increasingly rely on continuous sensing (video, loops, telemetry, incident feeds) to manage mobility, safety, and resilience. However, deploying machine learning in urban settings faces four persistent barriers:

1. **Edge constraints:** Real-time processing under limited compute, power, and bandwidth.
2. **Scale:** Many intersections, corridors, and sensors; large spatiotemporal dependency graphs.
3. **Adaptation:** Distribution shift due to construction, weather, events, and policy changes.
4. **Ethics and governance:** Privacy, compliance, and fairness must be measurable and enforceable.

We propose **E²-STH** (Edge-Efficient, Ethically-Constrained Spatiotemporal Hypergraphs): a modular architecture that (i) compresses raw sensing into a minimal state at the edge, (ii) learns a dynamic hypergraph model of multi-way urban interactions, (iii) produces forecasts and counterfactual scenarios, and (iv) enforces ethical constraints during training and deployment.

2. Problem Statement and Design Requirements

Given streaming observations from a set of sensors (e.g., cameras, telemetry) distributed across an urban region, we aim to:

- Forecast near-term urban variables (e.g., flow, occupancy, queue length, incident probability) over horizons H .
 - Generate counterfactual scenarios under interventions (signal timing changes, lane closures, event management strategies).
 - Provide decision support while meeting explicit requirements:
 - **Latency:** bounded end-to-end time per update.
 - **Resource limits:** bounded memory, power, and bandwidth usage on edge devices.
 - **Reliability:** graceful degradation and monitoring.
 - **Ethical compliance:** auditable constraints on privacy and operational policies, plus fairness constraints where applicable.
-

3. System Overview

The system consists of five modules:

1. **Perception & State Compression (Edge):** Converts raw video/sensor streams into compact features and summaries.
2. **Dynamic Hypergraph Builder:** Maintains a time-varying hypergraph capturing multi-way interactions.
3. **Spatiotemporal Hypergraph World Model:** Forecasts future states and uncertainties.
4. **Ethically-Constrained Learning & Inference:** Enforces constraints via a primal-dual mechanism.
5. **Counterfactual Scenario Engine:** Generates and evaluates interventions under constraints.

An optional sixth module provides a **hybrid quantum kernel** used only in attention/similarity scoring.

4. Methods

4.1 Perception & State Compression (Edge)

Goal: Replace raw high-dimensional signals with a privacy-preserving urban state x_t that supports downstream prediction.

Let o_t be raw observations at time t (video frames, telemetry, etc.). The edge module computes:

- **Object/event summaries:** counts, speeds, occupancy, queue length proxies, anomalies.
- **Region-level features:** motion/flow descriptors, density estimates.
- **Uncertainty estimates:** confidence intervals for extracted features.

We define the compressed state:

$$x_t = f_{\text{edge}}(o_t) \in \mathbb{R}^{|V| \times d}$$

where V indexes locations/regions (e.g., intersections, camera zones), and d is small.

Privacy-by-design: The edge module is configured to minimize retention of identifiable data (e.g., ephemeral processing, feature extraction without storing raw frames, optional redaction/blur pipelines, and cryptographic audit logs of retention policy).

4.2 Dynamic Hypergraph Construction

Urban systems often exhibit **multi-way interactions**: coordinated signal corridors, event-driven surges, spillback across multiple links, and coupled constraints.

We represent the city at time t as a directed hypergraph:

$$G_t = (V, \mathcal{E}_t)$$

Each hyperedge $e \in \mathcal{E}_t$ connects a **tail set** T_e to a **head set** H_e , enabling multi-to-multi relationships.

Hyperedges are constructed using: - **Topology rules**: road connectivity, corridor membership. - **Learned couplings**: data-driven correlations and causality proxies. - **Event overlays**: temporary hyperedges for incidents, weather alerts, planned events.

To avoid “hyperedge explosion,” we enforce: - Maximum hyperedge size $|T_e| + |H_e| \leq K$ - Maximum hyperedges per node $\leq M$ - Periodic pruning via stability and utility scores.

4.3 Spatiotemporal Hypergraph Neural World Model

We learn a forecasting model F_θ that predicts future states and uncertainty:

$$\hat{x}_{t+1:t+H}, \hat{\Sigma}_{t+1:t+H} = F_\theta(x_{t-L+1:t}, G_{t-L+1:t})$$

Hypergraph message passing. Let x_v be the feature vector at node v . For each hyperedge e , compute a hyperedge message:

$$m_e = \rho\left(\sum_{v \in T_e} W_T x_v, \sum_{v \in H_e} W_H x_v, z_e\right)$$

where z_e includes hyperedge metadata (type, duration, confidence). Node updates aggregate incident hyperedge messages:

$$x'_v = \psi\left(x_v, \sum_{e: v \in T_e \cup H_e} \alpha_{ve} m_e\right)$$

where α_{ve} are attention weights.

Temporal modeling. We use a gated recurrent module or temporal transformer over L steps, applied to node embeddings.

4.4 Ethically-Constrained Learning and Inference

Ethics must be operationalizable. We define a **constraint vector** $c(x_t, a_t) \leq 0$ that quantifies violations for:

- **Privacy:** retention, identifiability risk, prohibited attribute inference, sensitive location rules.
- **Policy compliance:** constraints from city/agency operating policies.
- **Fairness (when applicable):** parity constraints across protected groups or neighborhoods, measured on outcomes relevant to service quality.

We treat forecasting/decision support as constrained optimization. For a policy or decision rule π producing action $a_t = \pi(x_t)$:

$$\min_{\pi} \mathbb{E} \left[\sum_t \ell(x_t, a_t) \right] \quad \text{s.t.} \quad \mathbb{E}[c_j(x_t, a_t)] \leq 0 \quad \forall j$$

Primal-dual enforcement. Introduce Lagrange multipliers $\lambda \geq 0$:

$$\min_{\pi} \max_{\lambda \geq 0} \mathbb{E} \left[\sum_t \ell(x_t, a_t) + \lambda^\top c(x_t, a_t) \right]$$

Update θ (model/policy parameters) and λ online:

$$\theta \leftarrow \theta - \eta_\theta \nabla_\theta (\ell + \lambda^\top c), \quad \lambda \leftarrow [\lambda + \eta_\lambda c]_+$$

This provides a transparent mechanism: when measured violations rise, the system automatically penalizes violation-causing behaviors.

Auditability. Constraints and measurements are logged with versioned definitions, enabling third-party review.

4.5 Constrained Counterfactual Scenario Engine

We generate counterfactuals under interventions u (e.g., signal plans, closures). Let $\mathcal{M}$ be the world model rollout operator:

$$(x_{t+1:t+H}, \Sigma_{t+1:t+H}) = \mathcal{M}_\theta(x_t, G_t, u)$$

We search interventions by minimizing expected cost while respecting constraints:

$$\min_u J(u) = \mathbb{E} \left[\sum_{k=1}^H \ell(x_{t+k}, u) \right] \quad \text{s.t.} \quad \mathbb{E}[c(x_{t+k}, u)] \leq 0 \quad \forall k$$

Sampling/annealing. For discrete or mixed interventions, we use stochastic search (e.g., cross-entropy method or simulated annealing) with constraint-aware proposal distributions. Candidate rollouts violating constraints are penalized during sampling (not merely filtered after the fact) to prevent “loophole” solutions.

4.6 Optional Module: Hybrid Quantum Kernel for Attention

If a quantum component is required, we restrict it to a **single, benchmarkable subroutine**: computing similarity/attention scores.

Let u be a feature vector for a node or hyperedge context. A parameterized circuit defines $|\phi_\theta(u)\rangle$. The kernel is:

$$K_\theta(u, v) = |\langle \phi_\theta(u) | \phi_\theta(v) \rangle|^2$$

We use K_θ to define attention weights α_{ve} or hyperedge similarity.

Non-negotiables: - Quantum is **optional**; the classical system must meet requirements. - Quantum contribution must be supported by an ablation study: - Classical attention vs. classical kernel vs. quantum kernel. - Claims are limited to measured improvements (accuracy, latency, energy) under documented conditions.

5. Evaluation Protocol

5.1 Tasks

We evaluate on forecasting and decision-support tasks: - Forecasting: $x_{t+1:t+H}$ (flow, occupancy, queue proxies, incident probability). - Counterfactual evaluation: intervention u impact on travel time proxies, congestion risk, and safety indicators.

5.2 Metrics

- **Forecast accuracy:** MAE/RMSE, calibration (e.g., interval coverage), and event detection metrics for incident prediction.
- **Scenario quality:** constraint satisfaction rate, improvement in objective (e.g., reduced congestion proxy), robustness under noise.
- **Edge performance:** latency (ms), memory (MB), power (W), bandwidth (kbps).
- **Ethics/compliance:** well-defined violation metrics (see §6).

5.3 Baselines

- Spatiotemporal graph neural network without hyperedges.
- Temporal transformer over node features.
- Classical simulation-based heuristics for interventions.

5.4 Ablations (Required)

We report: 1. Baseline (no hypergraph) 2. + Hypergraph structure 3. + Constraint enforcement (primal–dual) 4. + Counterfactual engine 5. + Optional quantum kernel

5.5 Reproducibility

- Fixed data splits, public configuration files, versioned constraint definitions.
 - Deterministic seeds for evaluation runs.
 - Reporting of compute/hardware environment for edge benchmarks.
-

6. Ethics and Governance Specification

This section defines constraints as measurable quantities.

6.1 Privacy Constraints (Examples)

Define measurable privacy risk c_{priv} as a weighted sum of: - retention time of raw identifiable data, - identifiability score of intermediate representations, - sensitive inference flags (e.g., prohibited attribute inference), - access control violations.

We enforce:

$$\mathbb{E}[c_{\text{priv}}(x_t, a_t)] \leq 0$$

6.2 Policy Compliance Constraints

Constraints encode agency rules such as: - prohibited actions under specific alerts, - safe parameter ranges for signal changes, - maximum intervention frequency.

6.3 Fairness Constraints (When Applicable)

When decisions impact service quality across neighborhoods, define group metrics (e.g., average delay proxy by region) and enforce parity constraints with tolerance ϵ :

$$|\mathbb{E}[y \mid g = i] - \mathbb{E}[y \mid g = j]| \leq \epsilon$$

6.4 Audit Trail

All constraint computations are logged with: - metric version, - model version, - data window, - violation magnitude, - mitigation actions triggered.

7. Preliminary Results (Reporting Standard)

To prevent inconsistencies, we adopt a single reporting table per task with: - dataset version and split identifiers, - exact horizons H , context length L , and sampling frequency, - metric definitions, - mean $\pm$ std over N runs.

Placeholder Table (to be filled with verified results):

Model Variant	MAE ↓	RMSE ↓	Violation Rate ↓	Edge Latency ↓
Baseline				
+ Hypergraph				
+ Constraints				
+ Counterfactual				
+ Quantum Kernel (opt.)				

We will only report numbers produced by the protocol in §5 with complete provenance.

8. Deployment Plan

8.1 Edge–Cloud Split

- Edge: f_{edge} perception + state compression, privacy enforcement, minimal local alerts.
- Cloud/Regional: hypergraph maintenance, forecasting, scenario planning, dashboard.

8.2 Monitoring and Drift Adaptation

- Online drift detectors on feature distributions and forecast residuals.
- Safe adaptation: update schedules bounded by constraint budgets; rollback if violations rise.

8.3 Fail-Safe Behavior

If constraints cannot be satisfied (e.g., privacy system failure, sensor compromise), the system transitions to a safe degraded mode (e.g., minimal aggregated metrics, no individualized outputs, restricted interventions).

9. Risks and Mitigations

- **Hypergraph complexity:** cap hyperedge counts; prune by stability/utility.
 - **Constraint misspecification:** require stakeholder sign-off; maintain a versioned constraint library.
 - **Counterfactual misuse:** enforce governance (who can run what interventions), rate limits, and audit logs.
 - **Quantum overclaiming:** optional module only; benefits validated by strict ablation.
-

10. Conclusion

E²-STH provides a practical, deployable path to real-time urban intelligence: efficient edge processing, scalable spatiotemporal hypergraph modeling, and enforceable ethics via explicit constraints. Counterfactual planning enables decision support under uncertainty, while an optional quantum kernel module can be evaluated honestly as a narrow component.

Appendix A. Pseudocode (High-Level)

Algorithm 1: Online Update Loop 1. Receive observations o_t from sensors 2. Compute compressed state $x_t = f_{\text{edge}}(o_t)$ 3. Update hypergraph G_t (topology + learned/event hyperedges) 4. Forecast $\hat{x}_{t+1:t+H}$ and uncertainty 5. If interventions requested, run constrained scenario search to propose u 6. Compute constraint metrics $c(x_t, a_t)$ and update dual variables λ 7. Log metrics and decisions; emit outputs

Appendix B. Sheaf-Theoretic Consistency (Optional, Formal)

If desired, we can express multi-source consistency using sheaves: - Define a cover of the city by overlapping regions $\{U_i\}$. - Assign data sections s_i to each U_i . - Consistency is enforced by requiring restrictions agree on overlaps $U_i \cap U_j$.

This provides a principled lens for “global-from-local” consistency checks (e.g., detecting sensor conflicts). In this revision, sheaf concepts are used only where they provide measurable consistency tests.

Appendix C. Fastest Path to Validation (Checklist)

- [] Fix dataset specification and splits
- [] Implement ablation ladder (§5.4)
- [] Publish edge benchmark results (§5.2)
- [] Finalize constraint library and audit definitions (§6)
- [] Provide reproducible configs, seeds, and provenance logs